Diffusion Blocks

Trabajo integrado de Computación e IA

Jaime Etxebarria y Aimar Pagonabarraga

Abstract

La idea del proyecto es realizar el preentrenamiento de una LLM pequeña, tipo SmolLM2-135M utilizando un entrenamiento de next-token-prediction estándar (baseline) y compararlo con una versión entrenada utilizando la técnica descrita por DiffusionBlocks.

Arquitectura

Pequeño Transformer decoder-only autoregresivo, utilizando RMSNorm, RoPE, grouped-query attention, y SwiGLU, cuyas características principales son las siguientes:

Componente	**Tamaño**
Vocabulario	49.152
Representación de cada token	576
Capas Transformer	30
Cabezas de atención por capa	9
Cabezas de clave/valor por capa	3
Dimensión de cada cabeza de atención	64
Dimensión intermedia de la red feed-forward	1.536
Longitud máxima de contexto	8.192

Utilizamos la clase *LanguageModel*, que lee los archivos .yaml y crea las arquitecturas descritas tanto para el modo baseline como para el modo diffusion blocks.

El modelo baseline es bastante sencillo, y no dista de cualquier otra LLM. En cambio, el modelo diffusion blocks sí que tiene algunos cambios clave que merece la pena mencionar:

- Dentro del modelo, hay un stream "ruidoso" y otro "limpio". Al predecir el token i (hacer el denoising), se pueden utilizar los tokens limpios $0 \dots i-1$, como contexto. Para ello, utilizamos una máscara de atención, que indica qué tokens limpios puede ver cada token con ruido.
- A la hora de generar, cada bloque en el que está dividido la red hace varias pasadas sobre la entrada (configurable), reduciendo el ruido progresivamente en cada una. Por ejemplo, si el bloque 0 trabaja en el rango de ruido $\sigma = 5$ a $\sigma = 0.58$, hace las pasadas 5.0, 4.45, 3.89, 3.34, 2.79, 2.24, 1.69, 1.13, y 0.58.
- **adaLN-Zero:** Para que cada capa sepa con qué ruido está trabajando en cada iteración, hace falta pasarle σ. Para modular el comportamiento de cada bloque, utilizamos adaLN-Zero (adaptive layer norm). Cada bloque, recibe σ, crea un vector usando seno y coseno de $\log \sigma$ de distintas frecuencias, y luego pasa el vector por una red lineal + SiLU + lineal, generando un descriptor de ruido. Ese descriptor de ruido, a su vez, en cada capa de cada bloque, se pasa por una red lineal que genera 6 vectores para cada capa. Cada subcapa (RMSNorm + Attention o RMSNorm + MLP) utiliza 3 vectores, uno para escalar y otro para desplazar la entrada después de la normalización, y uno para controlar la *gate* de salida. Lo de "Zero", es porque la inicialización de la red lineal que genera los 6 vectores es a 0, por lo que inicialmente la entrada es igual a la salida, y se van añadiendo correcciones gradualmente para que el entrenamiento empiece suavemente.

- **EDM Preconditioning:** El valor de los valores de los embeddings varía muchísimo dependiendo del nivel de ruido. Para que cada bloque vea valores de tamaño similar, los embeddigns de la parte ruidosa se normalizan. Además, cuando el nivel de ruido es bajo, la entrada debería ser similar a la salida, y cuando el nivel de ruido es alto, la salida debería ser distinta. Definimos c_skip (cuanto de la entrada z nos quedamos) para que sea el ratio entre la varianza de ruido de una entrada limpia y la varianza de ruido de la que realmente tenemos (ruidosa). Además, definimos c_out (cuánta corrección hace falta), partiendo de z * c_skip. La salida de cada bloque, por lo tanto, es c_skip * z + c_out * F. Así, conseguimos que la salida de cada bloque sea siempre un valor en el mismo rango, y todos los niveles de ruido cuenten igual en el entrenamiento.
- **Cross-Entropy:** Para saber si el modelo está aprendiendo, es necesario medir cuánto se equivoca. El modelo asigna una probabilidad a cada token del vocabulario, y la cross-entropy calcula $-\log$ de la probabilidad que le ha dado al token correcto. Si le asigna una probabilidad alta, la pérdida es pequeña; si le asigna una probabilidad cercana a 0, la penalización es grande. Se obtiene el valor sobre los tokens válidos y lo utilizamos para entrenar ambos modelos: el baseline aprende a predecir el siguiente token, y diffusion blocks aprende a recuperar el token correcto a partir de su versión ruidosa y del contexto limpio anterior.

Proceso de entrenamiento

Antes de comenzar con el entrenamiento de la arquitectura objetivo sobre el dataset completo, se han realizado 4 pruebas: A. B, C y D. La prueba A consiste en un testeo rápido de la carga de datos, entrenamiento, evaluación, guardado del checkpoint y generación ejecutado en CPU, simplemente para comprobar que el pipeline funciona correctamente. Esto se ha realizado en un modelo de prueba reducido de 6 capas y utilizando únicamente 8 historias de entrenamiento, 2 de validación y 12 pasos de actualización.

De forma similar, se ha realizado un prueba B con esa misma arquitectura, repitiendo las mismas 8 historias durante 300 actualizaciones, cuyo requisito único para pasarlo era que la caida de la pérdida de entrenamiento fuera al menos del 2 %, y poder confirmar así que el modelo está aprendiendo, aunque solo fuera por hacer overfitting de los mismo ejemplos de entrenamiento.

La prueba C fue definida en un comienzo para testear el funcionamiento en T4 de Google Colab. Se ha intentado ejecutar el código en Colab, pero finalmente no ha sido posible debido a múltiples errores de incompatibilidad de las versiones de las librerias y la incapacidad de vencer las limitaciones del entorno. Es por eso que finalmente se ejecutó la prueba D en la que se usaron 50.000 historias de entrenamiento y 500 de validación. Se mantuvo la anchura de 576 valores pero también se redujo la profundidad del modelo a 12 capas, divididas en 3 bloques de 4 capas cada uno. La ejecución recogió 3000 pasos con un batchsize efectivo de 16 y una longitud de 256 tokens. Esta fase se ha ejecutado en un RTX 2070.

Finalmente, habiendo comprobado que todo funciona correctamente, se ha realizado un entrenamiento completo de la arquitectura objetivo final (modelo SmolLM2 con 30 capas) usando el dataset completo (200.000 historias de entrenamiento, 2000 de validación y 2000 de test). Se ha entrenado una epoch con 246.326 ventanas de entrenamiento con secuencias de 256 tokens y un batchsize de 32, dando un total de 7698 actualizaciones del optimizador por modelo.